OCL: Object Constraint Language

Prof. Dr. Daniel Riesco

Introducción

Estandar de la OMG e ISO 19507:2012 (revisada y confirmada en el 2022).

- Usado para construir modelos exactos UML/MOF
  - Lenguaje pequeño
  - Tipos Simples built-in
  - Tipos de Colecciones
  - Usa elementos del modelo

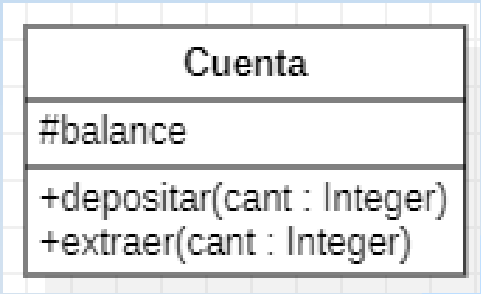
Introducción

- Describe restricciones lógicas
- Asociado a elementos del modelo
- Con proposito de documentación
- Usado para verificación formal
- Usado para generación de testcases
- Puede ser para chequeo en tiempo de ejecución
  - Puede producir excepciones
  - Diseño por contrato

OCL: Introducción (Pre y Pos condiciones)

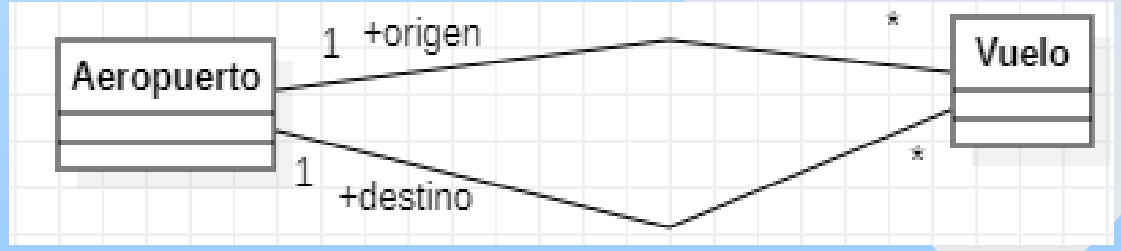
Especificar pre y poscondiciones en las operaciones

```
context Cuenta::depositar(cant : Integer)
    pre: cant >= 0
    post: balance = balance@pre + cant
context Cuenta::extraer(cant : Integer)
    pre: cant >= 0 and balance >= cant
    post: balance@pre – cant = balance
```



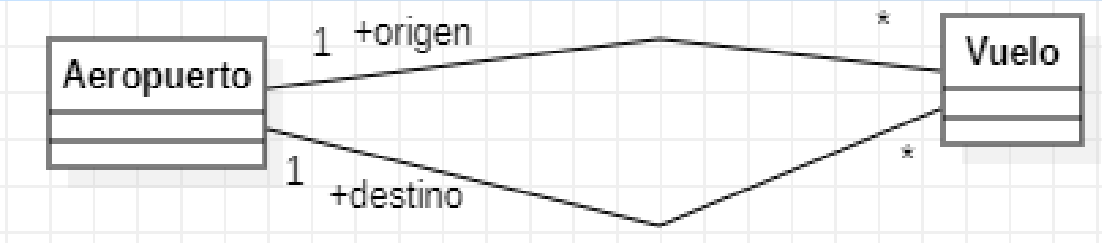
OCL: Introducción (Invariantes)

- Los invariantes son válidos durante la vida del objeto
- Restricciones entre elementos del modelo
  1) Definidos en el nivel del metamodelo
  2) Deben valer antes y después de la transformación



OCL: Introducción (Invariantes)

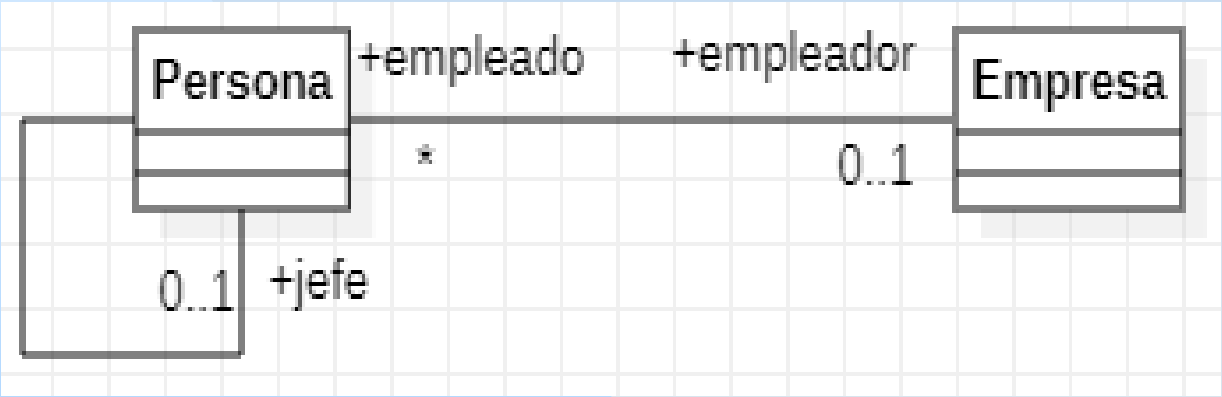
- Los invariantes son válidos durante la vida del objeto



context Vuelo
inv: self.origen <> self.destino

OCL: Introducción (Navegar)

- OCL para expresiones de navegación



OCL: Introducción (Navegar)

- OCL inicialización de código
- OCL para expresiones de navegación

```
context Copy
inv: self.copyOfBook.authorsOfBook->size() > 0
```



Introducción (5/5)

- Lenguaje formal para expresar restricciones libres de efectos colaterales.
- No es un lenguaje de programación (no hay flujo de control).
- No se puede invocar procesos.
- Tiene tipos.
- Propósitos:
  - especificar invariantes
  - especificar pre y post condiciones en los métodos
  - como lenguaje de navegación
  - especificar restricciones en las operaciones
    - operación(a, b, c) = expresión

Características de OCL

- Fácil de usar
  - Legible y escribible por una amplia audiencia
- Lenguaje de expresión pura
  - Libre de efectos colaterales
- Lenguaje de Especificación
  - Declarativo e independiente de la plataforma

Componentes

- Self
- Operaciones

  Context nbreTipo :: nbreOp(p1:T1, ...) : tipoRetorno

  pre: p1 ...

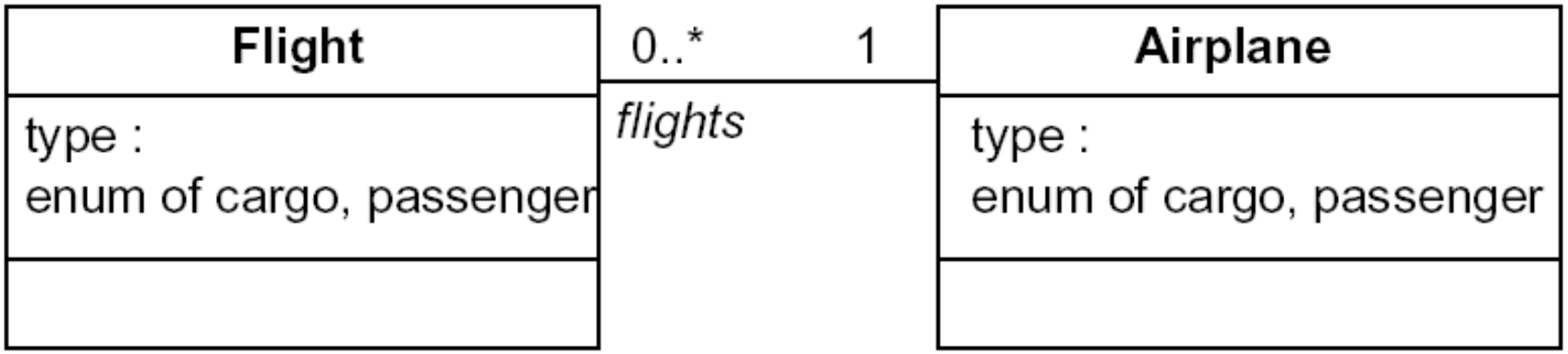
  post: result = ...
- Invariantes
  - Context ElementoModelo
  - inv: <expresionOCL>
  - Tipos básicos:
  - Boolean: and, or, not, implies, if-then-else
  - Integer: *, +, /, abs, mod, min, max, ...
  - Real: *, +, /, floor, <, ...
  - String: toUpper, concat, size, substring(i:integer, s:integer),...

Propiedades

- un extremo de una asociación (AssociationEnd)
  - Toda asociación en el modelo es un camino de navegación
  - El contexto de la expresión es el punto de comienzo
  - Los nombres de roles son usados para identificar el camino de navegación.
- AssociationEnd
  - cardinalidad 1 -> retorna un objeto o un set de un elem.
  - cardinalidad n -> retorna un set
  - cardinalidad n {ordered} -> retorna una sequence
- Reglas de Negocio
  - Las personas casadas son mayores de edad
  - Una persona con cargo directivo debe trabajar sólo en una empresa.
  - La remuneración mas alta no puede superar el 10% a la del Gerente General

Invariante

- Context <clasificador> inv: <exp-bool>

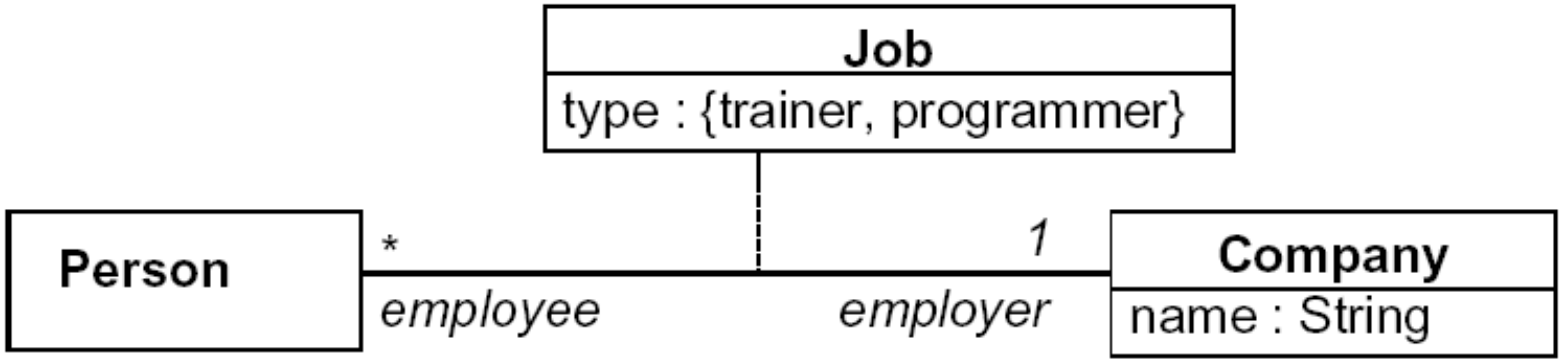


{context Flight
inv: type = #cargo implies airplane.type = #cargo
inv: type = #passenger implies airplane.type = #passenger}

Invariante – Clase Asociativa

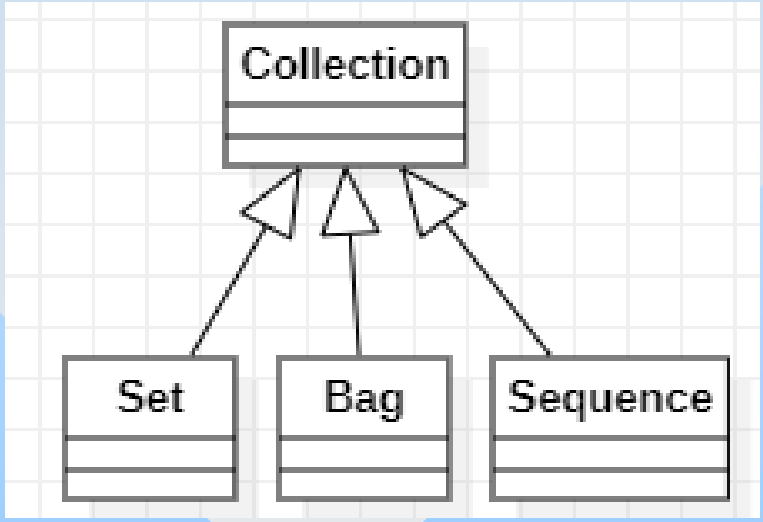
- Context Person : se usa el punto y el nombre de la asociación
- Context Job : hacia los objetos que participan

```
context Person inv:
if employer.name = 'Klasse Objecten' then
  job.type = #trainer
else
   job.type = #programmer
endif
```



Propiedades

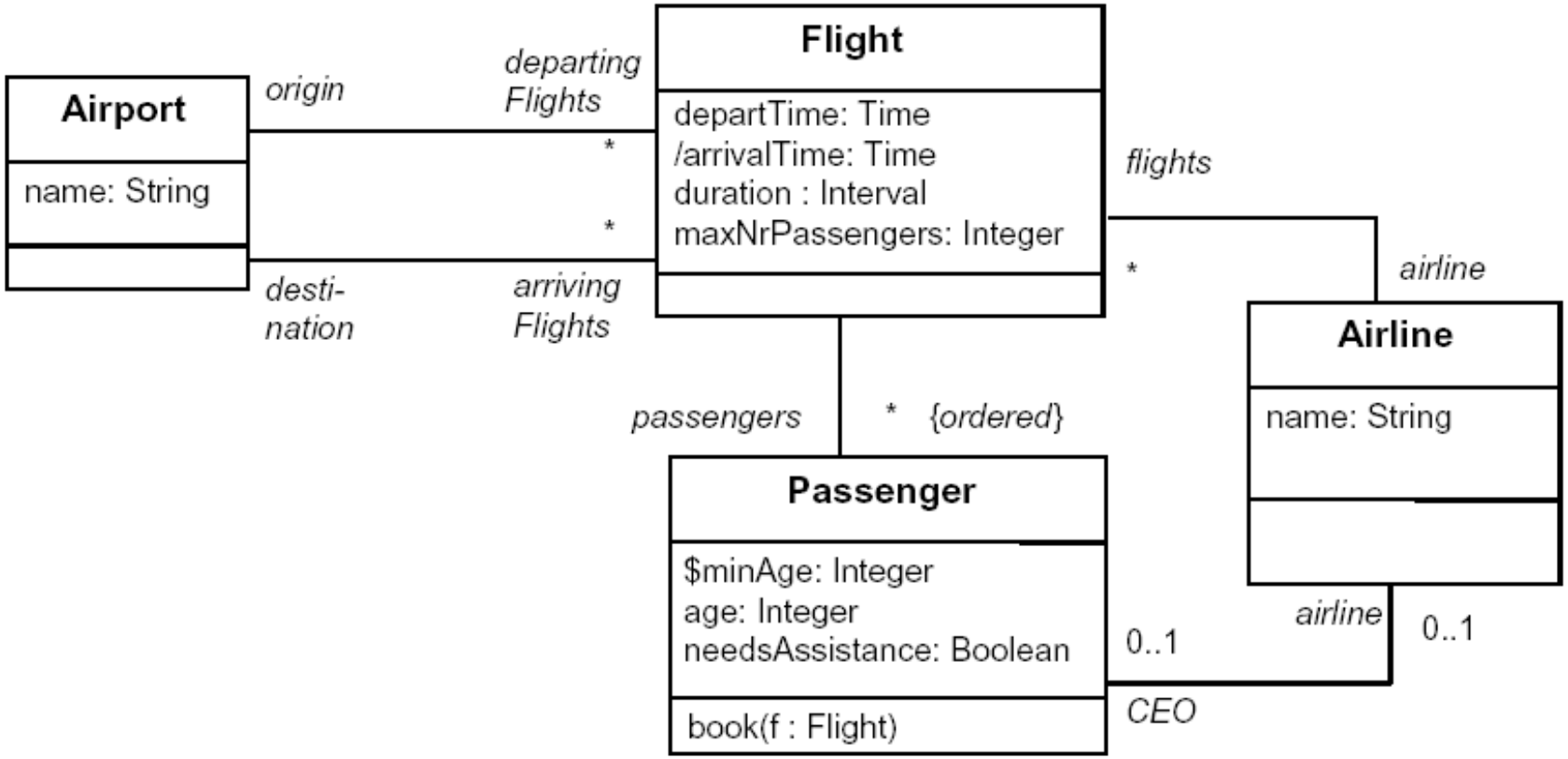
- Predefinidos:
  - =, <>, @pre
  - allInstance
    - Ej. Person.allInstance->forAll(p1, p2 | p1<> p2 implies p1.DNI <> p2.DNI}
- Colecciones
  - Set {1, 5, 7}
  - Sequence {1, 3, 45, 2}
  - Bag {1, 1, 4}



Operaciones de Collection

- Operaciones:
  - collection->select(v : type | boolean-expr-with-v)
  - collection->reject(v : type | boolean-expr-with-v)
  - collection->collect(v : type | expr-with-v)
  - collection->forAll(v : type | boolean-expr-with-v)
  - collection->exists(v : type | boolean-expr-with-v)
- Otras operaciones son:
  - isEmpty : verdadero si la colección no tiene elementos
  - notEmpty : not isEmpty
  - size : el número de elementos de la colección
  - count(elem) : número de ocurrencias de elem en la colección
  - include(elem) : verdadero si elem está en la colección
  - exclude(elem) : not include(elem)
  - includeAll(col) : verdadero si todos los elementos de col están en la colección
  - sum : suma los elementos de la colección (debe poder aplicarse el operador +)

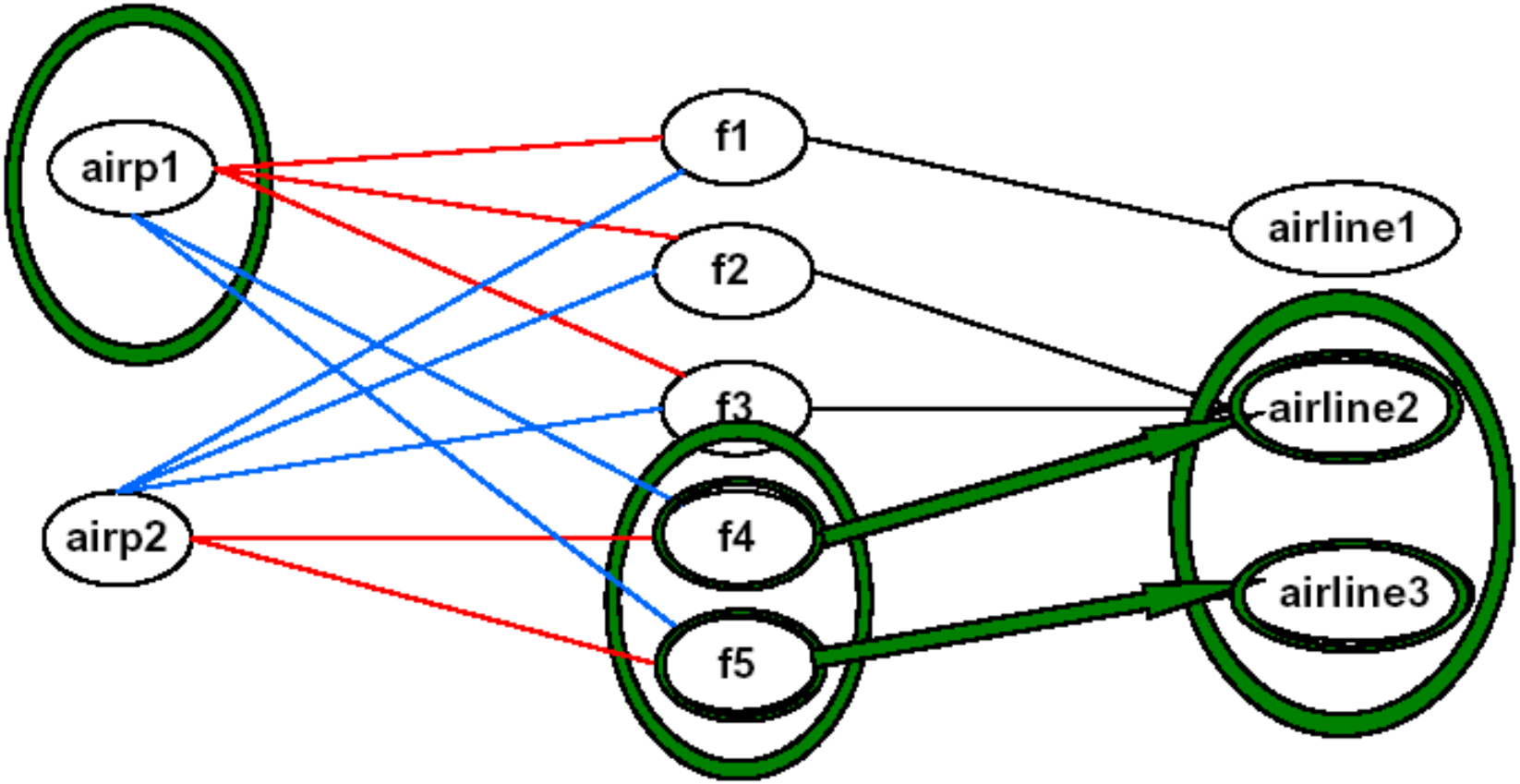
Ejemplo


Airport
name: String
origin
departing Flights
*
*
destination
arriving Flights
Flight
departTime: Time
/arrivalTime: Time
duration : Interval
maxNrPassengers: Integer
flights
*
airline
Airline
name: String
passengers
* {ordered}
Passenger
$minAge: Integer
age: Integer
needsAssistance: Boolean
book(f : Flight)
0..1
CEO
airline
0..1

Collection

context Airport inv:

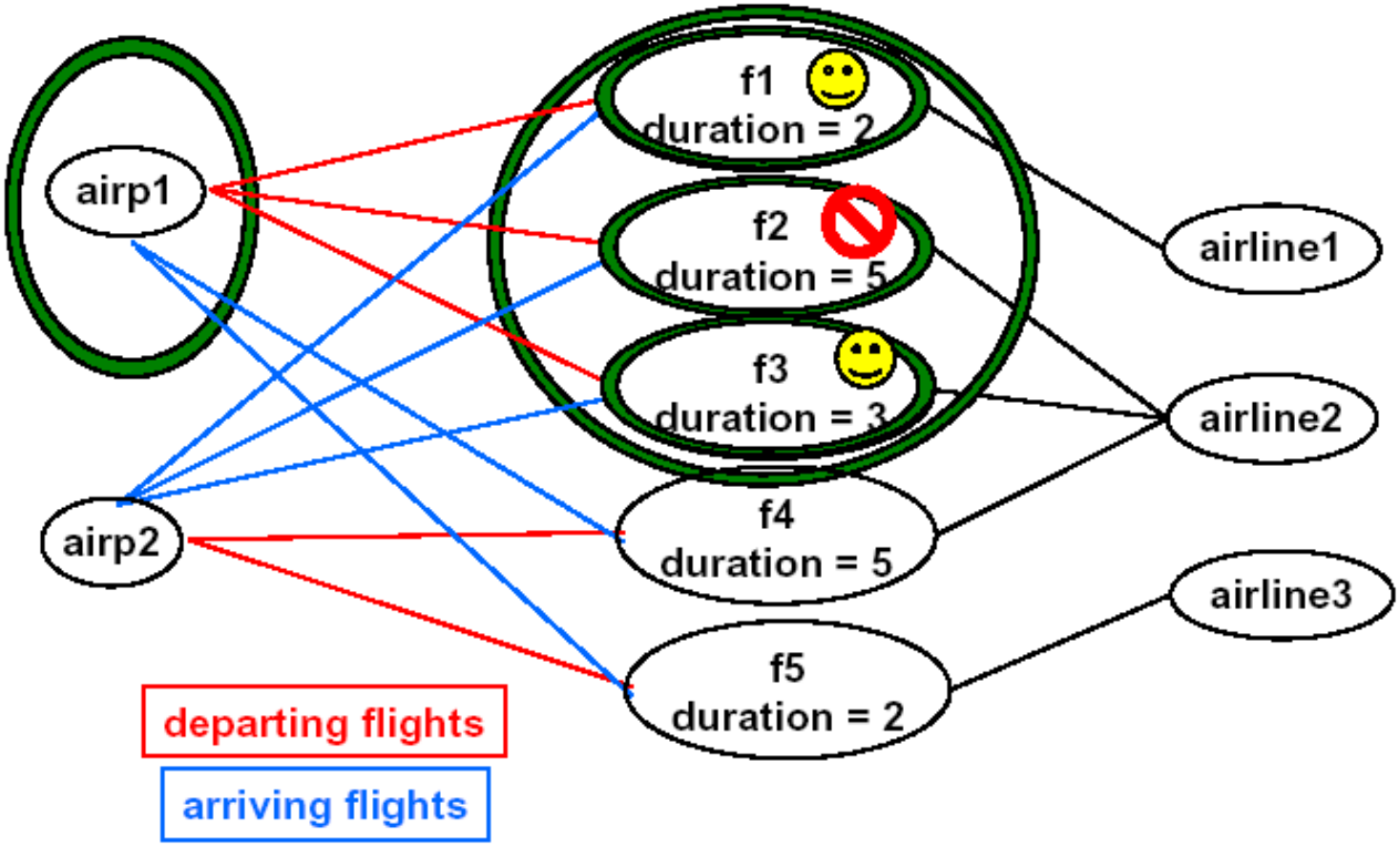
self.arrivingFlights->collect(airLine)->notEmpty



Select

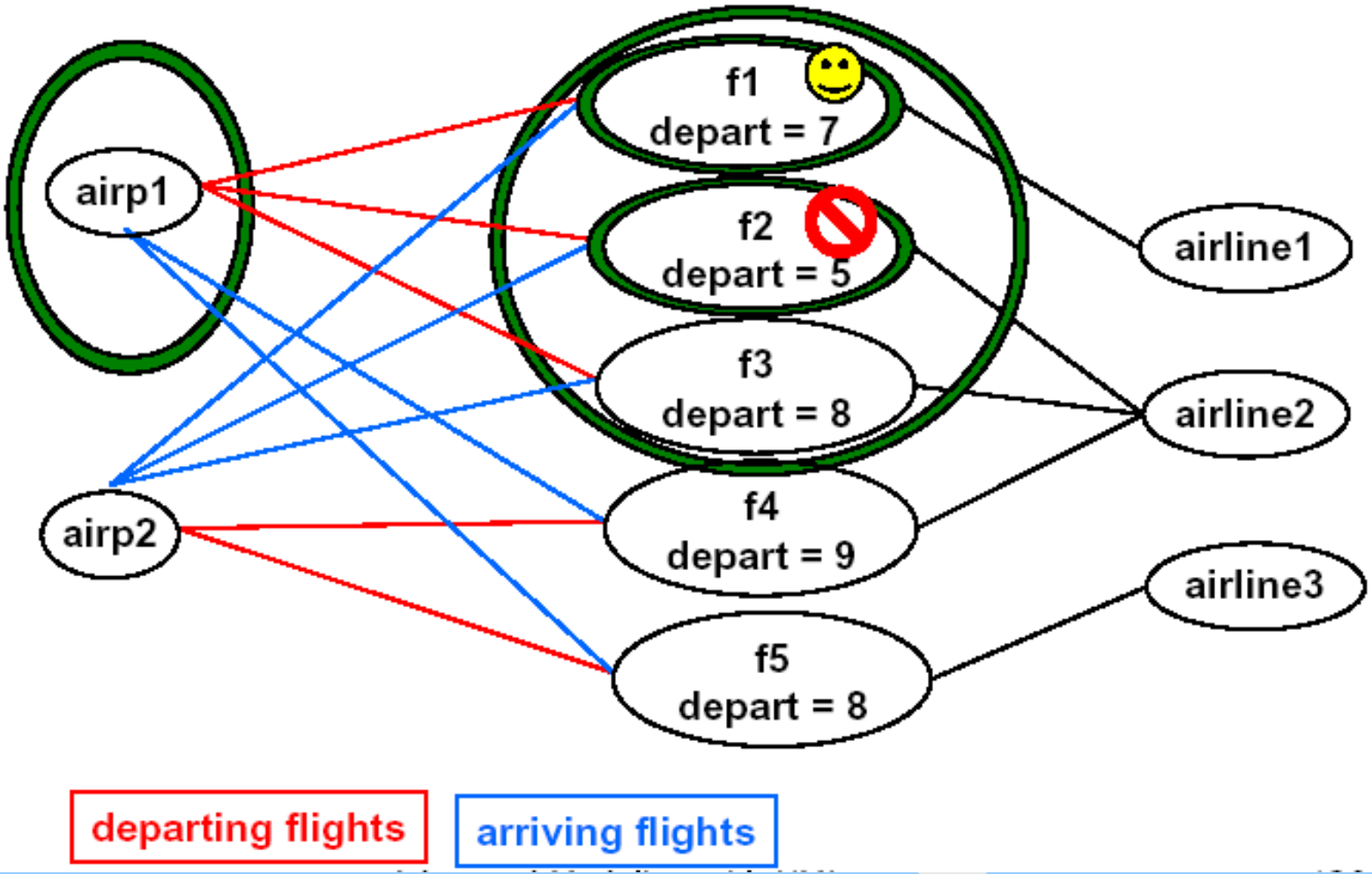
context Airport inv:

self.departingFlights->select(duration<4)->notEmpty



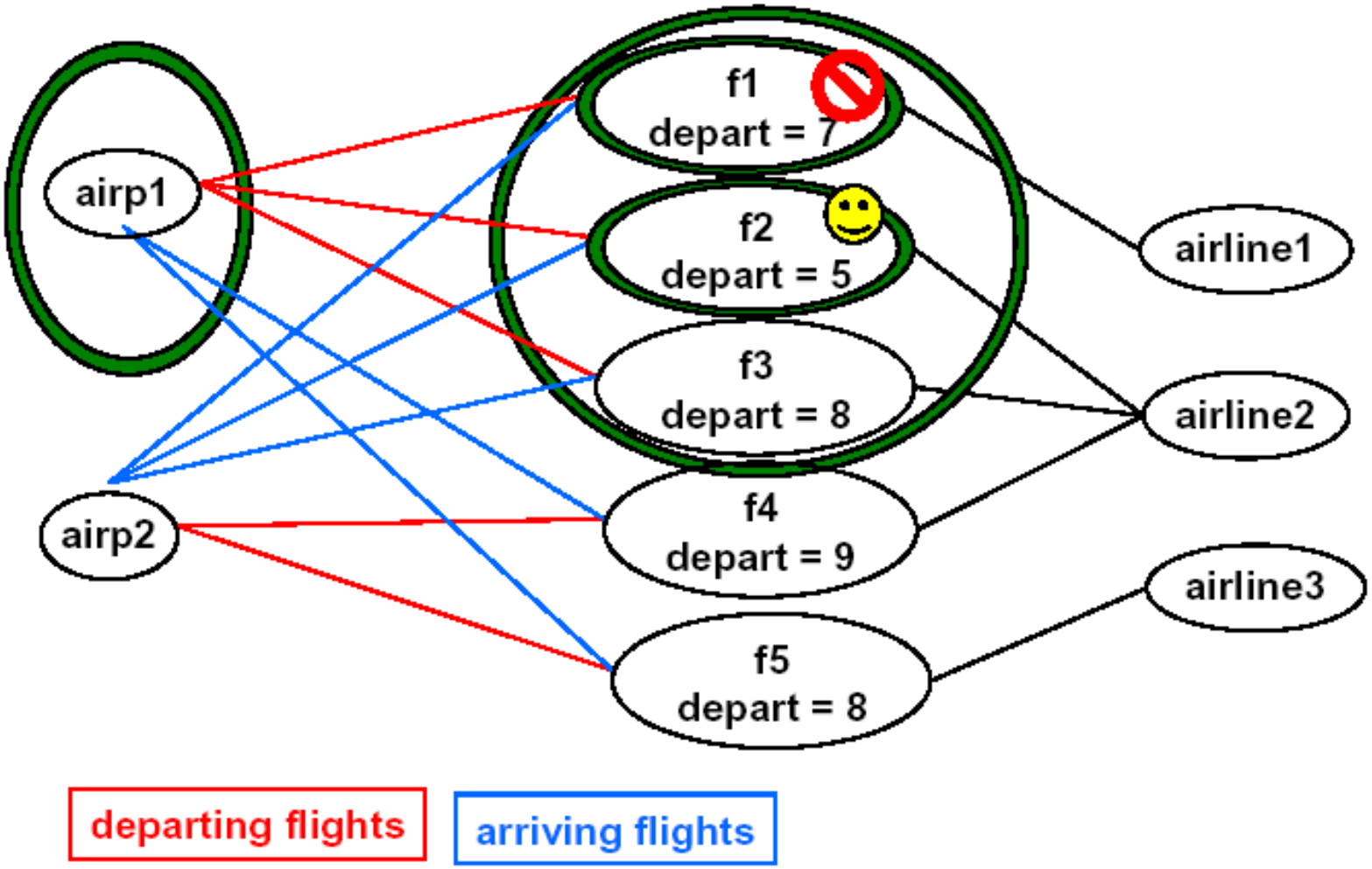
forAll

```
context Airport inv:
self.departingFlights->forAll(departTime.hour>6)
```



exists

```
context Airport inv:
self.departingFlights->exists(departTime.hour<6)
```



Set

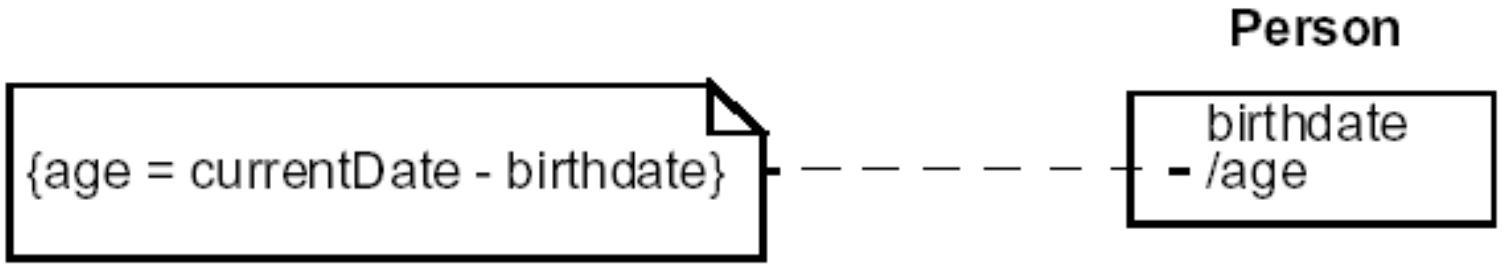
- set->union(set2 : Set(T)) : Set(T)
- set->union(bag : Bag(T)) : Bag(T)
- set->intersection (con set and bag)
- set->including(obj : T) : Set(T) (Agrega obj a set)
- set->excluding(obj : T) : Set(T) (Elimina obj a set)
- set->symmetricDifference(set2 : Set(T)) :Set(T)
- set->asSequence : Sequence(T) (orden aleatorio)
- set->asBag : Bag(T)
- Y todas las heredades de collect

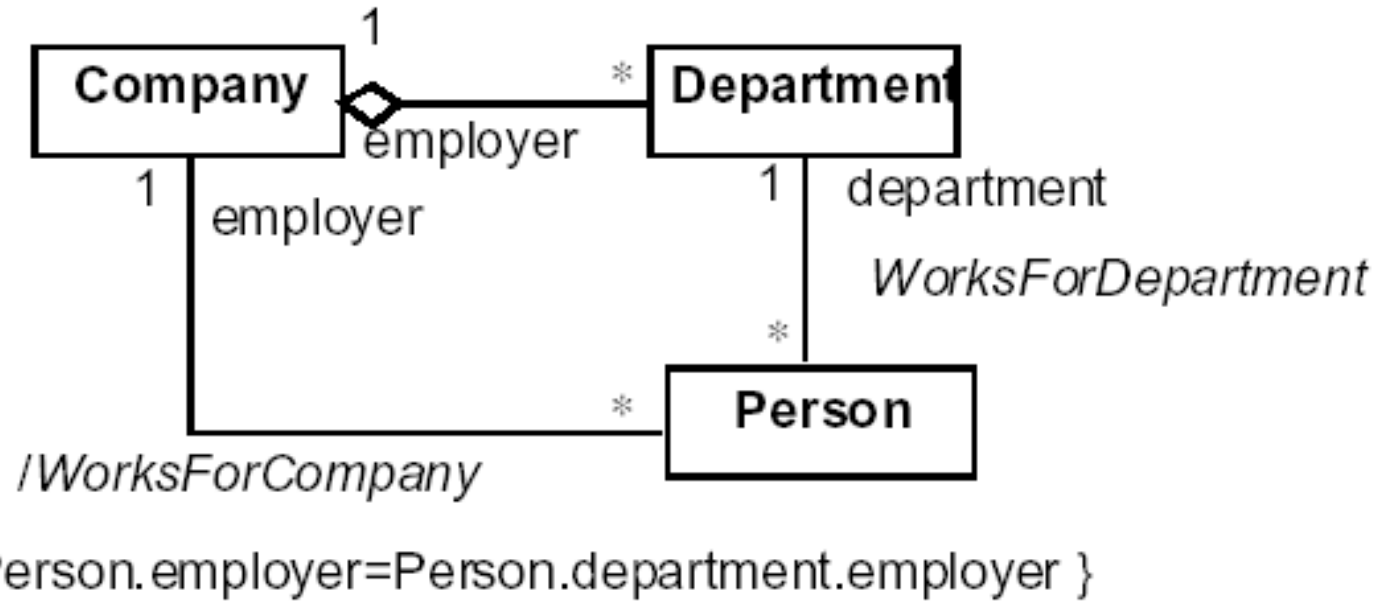
  =, -, select, reject, collect

Bags and Sequences

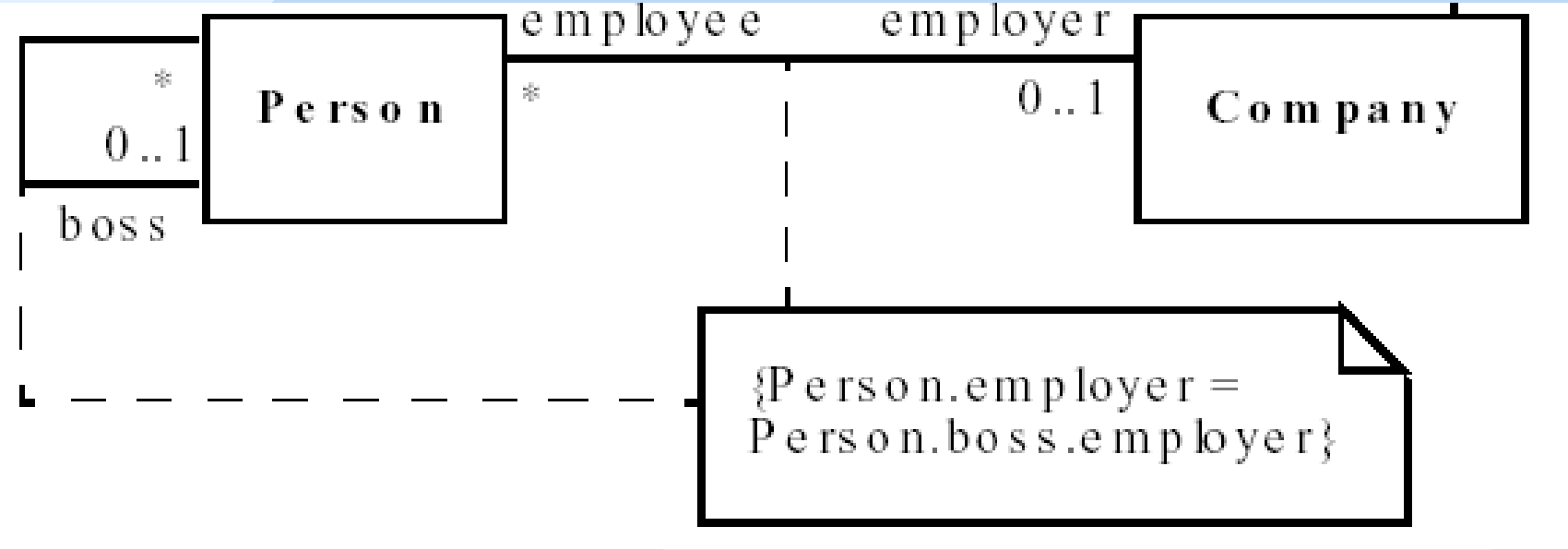
- Operaciones de una bolsa (Bag)
  - =, union, intersection, including, excluding, select, reject, collect, count, asSequence, asSet
- Operaciones de una Secuencia (Sequence)
  - count, =, union (es una concatenación), append (agrega el obj al final), prepend (agrega obj al principio), subSecuence(lower:integer, upper:Integer), at(i :Integer), first, last, exlcuding (quita todas las ocurrencias de obj), select, reject, collect, iterate, asBag, asSet

Atributos y Relaciones Derivadas





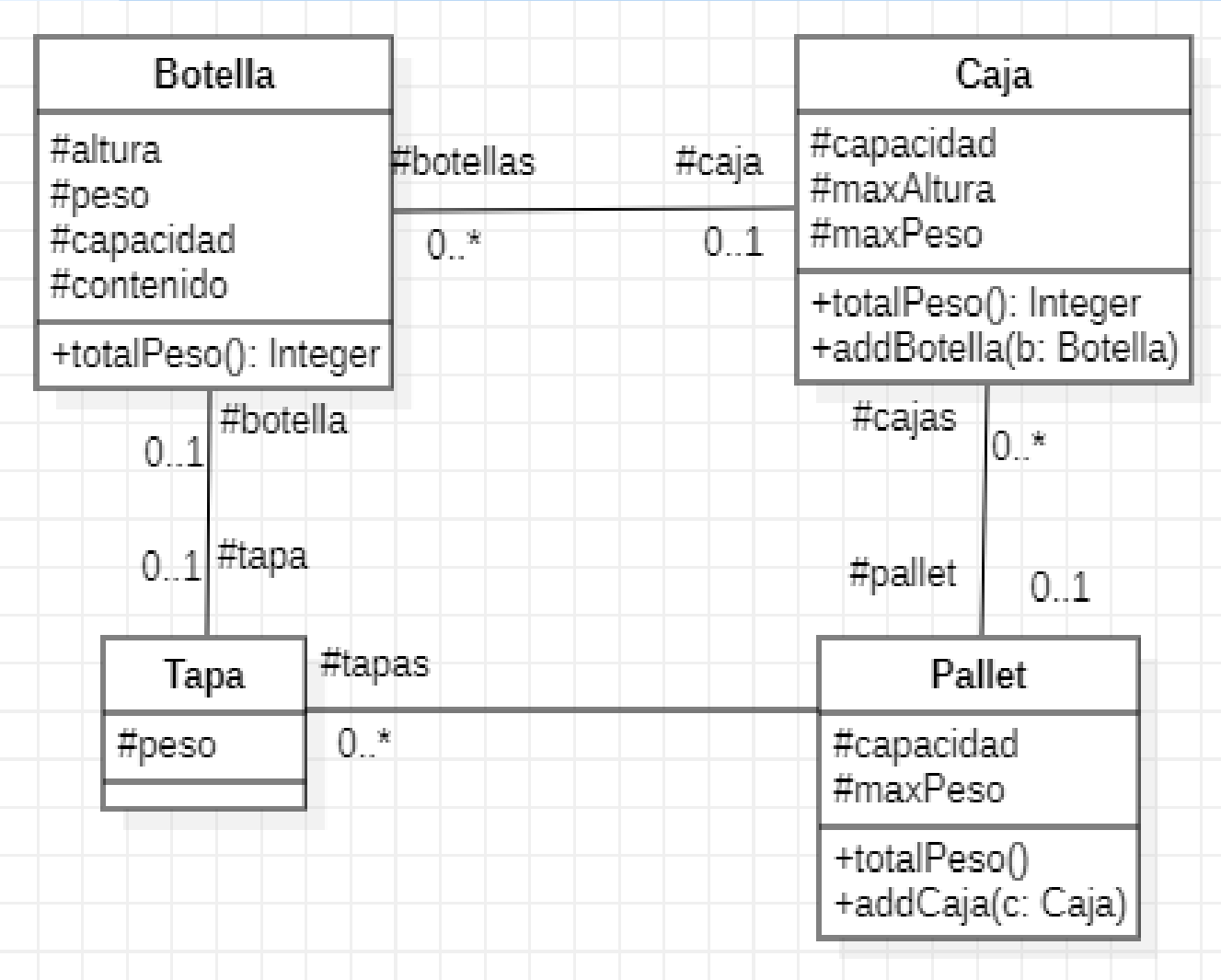
Restricciones



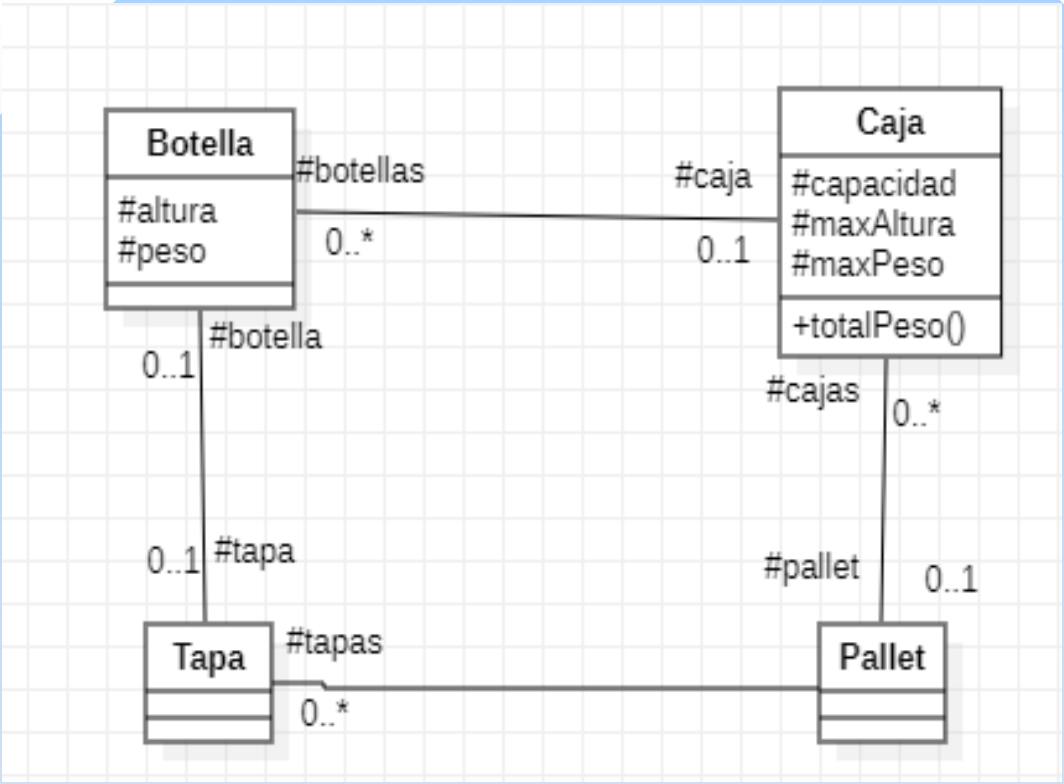
Otras propiedades

- Cast
  - t1 --> t2 cdo son = o t1 es subtipo de t2
  - Collection(t1) --> Collection(t2) cdo t1 es subtipo de t2
  - es transitivo.
- Valor previo en postcondición: nbreAtrib@pre
- OclType
  - type.name : String
  - type.attributes : Set(String)
  - type.associationEnds : Set(String)
  - type.operations : Set(String)
  - type.supertype : Set(OclType)
  - type.allSuperTypes : Set(OclType)
  - type.allInstance : Set(type)
- Let var : Type = <expression1> in <expression2>

Modelo de Dominio



Invariantes sobre las clases



context Caja

-- el número de botellas no puede exceder su capacidad

inv: botellas->size() <= capacidad

-- cada botella debe caber en la caja

inv: botellas->forAll(b | b.altura < maxAltura)

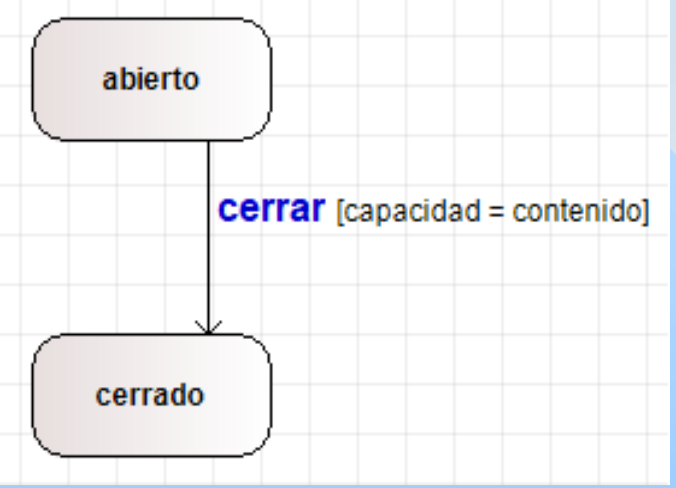
-- el peso total debe ser inferior al máximo

inv: totalPeso() <= maxPeso

Invariantes y Guardas de Estados

context Botella

state cerrado inv: contenido = capacidad

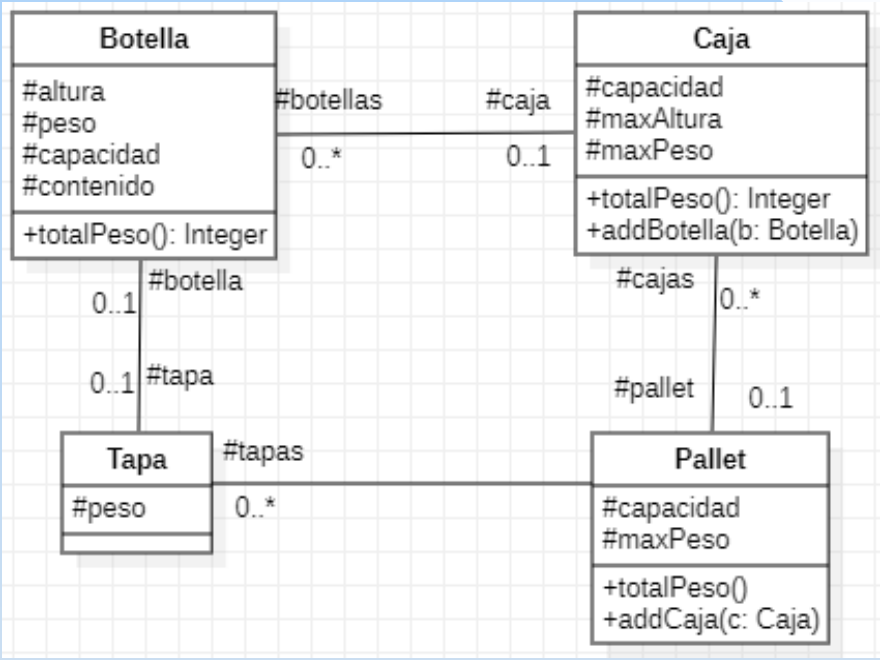




Design by Contract

- Pre- and post-conditions
- Takes “Design by Contract” principle into “Analysis by Contract”

Pre y poscondiciones sobre las operaciones



context

Caja::addBotella(b : Botella)

-- debe haber espacio en la caja

pre: **botellas->size() < capacidad**

-- No se puede exceder el peso máximo de la caja.

pre: **totalPeso() + b.totalPeso() <= maxPeso**

-- la botella no está en la caja

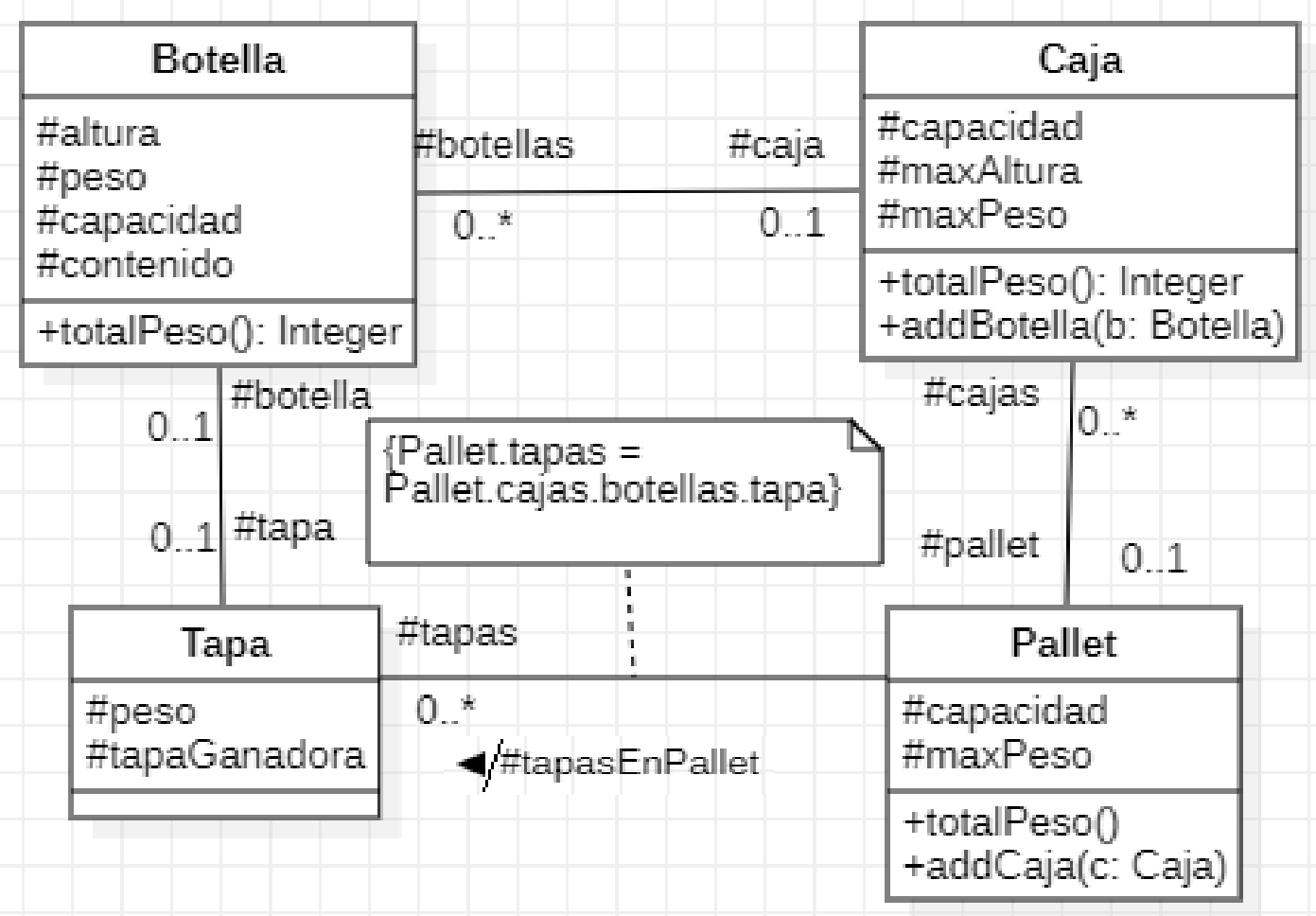
pre: **not botellas->includes(b)**

-- 'b' se agrega a la colección de botellas.
post: **botellas@pre->including(b) = botellas**

Use Case Pre/Postcondtions

- A use-case can be viewed as a "system function".
- Use case *assumptions* and *results* can be specified in OCL.

Relaciones Derivadas



Components

- Specified by an (or more) interface(s)
  - Operations with pre- and postconditions
  - Invariants on the objects described by the interface
    - Show business rules for the component
- OCL 2.0 Message clause
  - Specify user visible side-effects

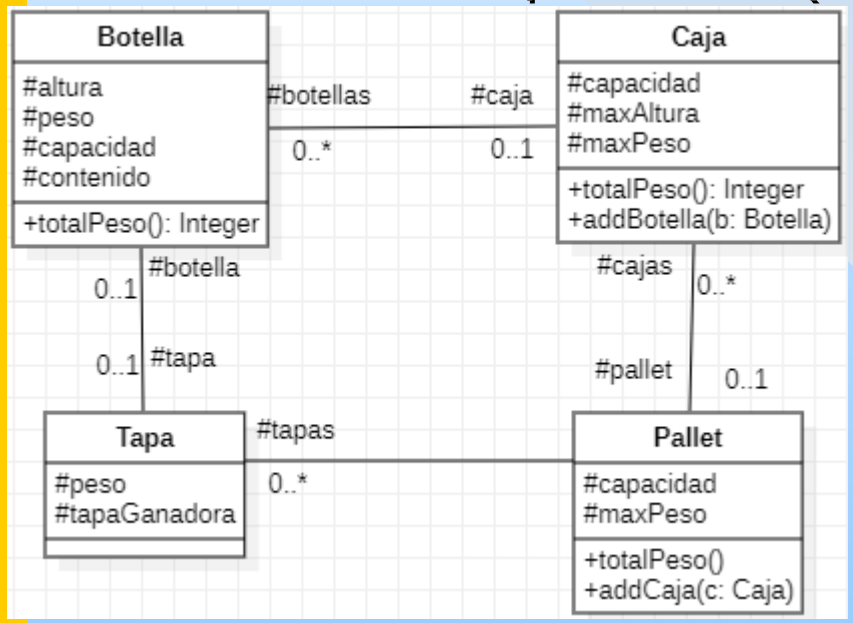
Reglas de Negocio

En un pallet puede haber como máximo tantas tapas con marcas ganadoras sobre el pallet como cajas en el pallet hay:

context Pallet inv:

tapas->select(tapaGanadora = true)->size()

<= caja->size()

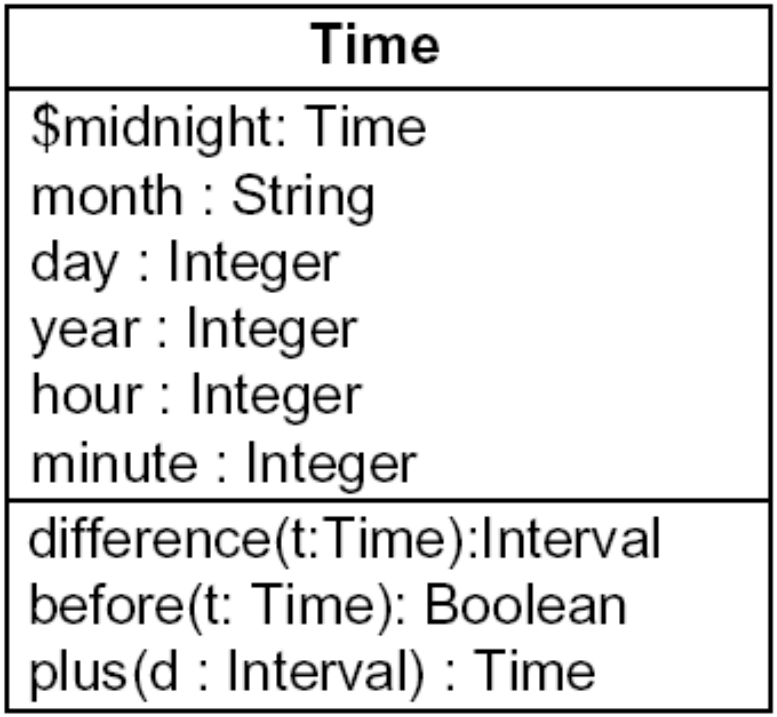


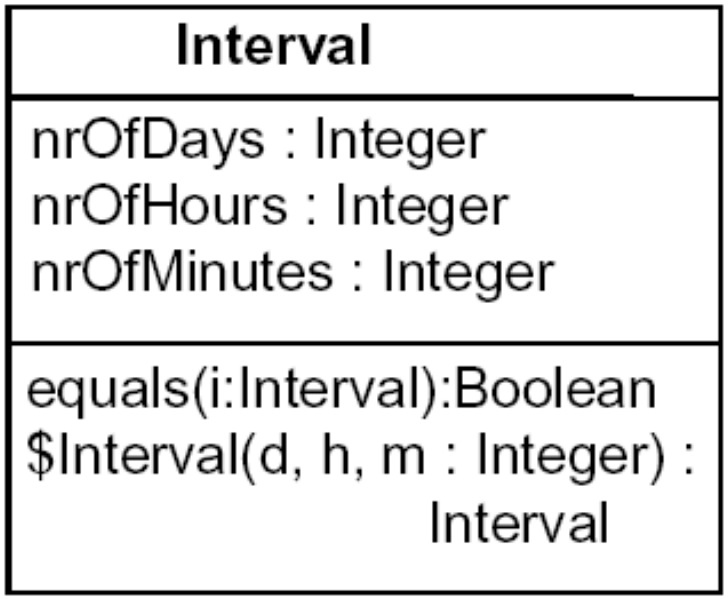
Queries

- OCL is a Query Language
  - Expressive power >= SQL
    - David Akehurst, Canterbury University
- Boldsoft
  - Uses OCL exclusively as a query language to specify the user interface
- Queries can be defined on the model level

Invocación de Métodos

```
context Flight inv:
self.departTime.difference(self.arrivalTime)
  .equals(self.duration)
```





Principio de Sustitución de Liskov

- "En cualquier lugar que una instancia de una clase es esperada, uno puede siempre sustituirla por cualquiera de sus subclases"
- Consecuencias para los invariantes:
  - Un invariante es siempre heredado por cada uno de las subclases donde dicho invariante es ubicado
  - Las subclases pueden fortalecer dicho invariante
- Consecuencias para las pre y post condiciones
  - Una precondición puede ser más débil
  - Una poscondición puede ser más fuerte

Resumen

- Permite lograr especificaciones precisas y no ambiguas en modelos UML
- Permite especificaciones sin usar lenguaje natural
- Métodos formales son díficiles para el modelador de negocios o de sistemas promedio. OCL ha sido desarrollado para cubrir esa brecha.

Resumen

- OCL permite:
  - Especificar invariantes sobre distintos elementos de los modelos
  - Especificar guardas en los diagramas de estado
  - Especificar atributos y relaciones derivadas
  - Especificar pre y poscondiciones en operaciones

Bibliografía

Jos Warmer, Anneke Kleppe, “Object Constraint Language, The: Getting Your Models Ready for MDA”, Second Edition, 2003, Pearson Education.

https://www.omg.org/spec/OCL/2.4

Raul Sidnei Wazlawick, “Object-Oriented Analysis and Design for Information Systems Agile Modeling with BPMN, OCL, IFML, and Python”, Second Edition, 2024, Elseiver.

Ejemplos del Metamodelo

- Asociación
  - Los finales de asociación deben tener un nombre único dentro del final de Asociación.
  - A lo sumo un final de asociación puede ser una agregación o composición.
- Final de Asociación
  - El clasificador de un final de asociación no puede ser un DataType ni una Interface.
  - Una instancia es de composición no puede pertenecer a más de una instancia compuesta.
- Clasificador
  - No puede haber dos métodos con la misma signatura.
  - No puede haber dos atributos con el mismo nombre